Astana IT University

School of Software Engineering

PRE-DEFENSE 1 REPORT

Development of a System for Comprehensive Analysis and Evaluation of Scientific Information Using Digital and Intelligent Technologies

Students: Saltanat Sergeyeva, Alzira Kuanyshbekova, Meruert Khairgeldinova

Supervisor: Almas Myrzakhmetov

Astana, 2026

Project Card

Topic (RU)	Разработка системы комплексного анализа и оценки научной информации с использованием цифровых и интеллектуальных технологий
Supervisor	Almas Myrzakhmetov, Senior Lecturer, School of Software Engineering
GitHub repository	https://github.com/Akua49/scientific-analysis-system
YouTrack	https://sergeyevasss.youtrack.cloud/projects/SCI
Prototype	https://www.figma.com/proto/Gu5Z6gvbep6zOqVz4ASxwa/Scirevia-%E2%80%94-Prototype?node-id=9-150&t=EZQoz7IviUS2yC53-1

Team and contribution

Member	Role	Area of responsibility	GitHub account
Saltanat Sergeyeva (SE-2403)	ML / Data engineer	Analysis service: text preprocessing, citation verification, summarisation, topic similarity, intelligent part of requirement checks, metadata from OpenAlex / Crossref / Semantic Scholar, quality evaluation	@ssaltuwwa
Alzira Kuanyshbekova (SE-2403)	Frontend developer, UX/UI designer	Scenarios and prototype, upload and check launch, requirements window, report visualisation, API integration	@Akua49
Meruert Khairgeldinova (SE-2414)	Backend developer	Server architecture and REST API, document upload and text/reference extraction, link checking (URLs, redirects, DOI), storage, business logic, integration with analysis service	@tw3tyy
Supervisor: Almas Myrzakhmetov, Senior Lecturer, School of Software Engineering	Supervisor	Consultation on architecture, task setting, documentation; review at checkpoints	—

Selected technologies

Layer	Technology
Client	HTML, CSS, JavaScript (ES modules), Chart.js; design in Figma
Server	Node.js, Express.js, MongoDB (Mongoose), REST API + OpenAPI (Swagger)
Analysis service	Python, FastAPI, spaCy / NLTK, scikit-learn, Hugging Face Transformers (multilingual sentence embeddings, NLI), Pandas; LLM API optional
External sources	Crossref, OpenAlex, Semantic Scholar API, doi.org
Infrastructure	Docker Compose, GitHub (+ Actions), YouTrack

1. Project Proposal

Field	Value
Title (EN)	Development of a System for Comprehensive Analysis and Evaluation of Scientific Information Using Digital and Intelligent Technologies
Title (RU)	Разработка системы комплексного анализа и оценки научной информации с использованием цифровых и интеллектуальных технологий
Short name	Scirevia
Programme	Software Engineering, AITU
Team	Saltanat Sergeyeva (ML/Data), Alzira Kuanyshbekova (Frontend, UX/UI), Meruert Khairgeldinova (Backend)
Supervisor	Almas Myrzakhmetov, Senior Lecturer, School of Software Engineering
Client / data provider	School of Software Engineering, AITU (archive of past thesis topics, control sample) — contact through the supervisor, A. Myrzakhmetov

1.1 Problem statement

When diploma theses, projects, and scientific articles are prepared, the same problems recur:

- links in the reference list do not open or lead to a different source;
- quoted or paraphrased text does not correspond to the text of the cited source;
- the abstract does not reflect the actual content of the work;
- the document does not meet the established requirements;
- the topic repeats theses of previous years.

Today, supervisors and norm-control staff check these manually, which takes considerable time and depends on the reviewer. Plagiarism systems detect copied text but do not check whether citations and references are *correct*.

1.2 Aim

To develop a web system that automates checking references, citations, abstracts, user-defined requirements, and topic originality in scientific texts using NLP and open scholarly data, and produces a single report for the author and reviewer.

1.3 Objectives

1. Collect and verify requirements through an online survey of students and academic staff (S-1, N = 26) and follow-up interviews with supervisors and norm-control staff; obtain the archive of past topics from the School.
2. Review literature on reference rot, citation accuracy, reference parsing, semantic similarity, summarization, and factual-consistency evaluation.
3. Compare existing tools (Turnitin, StrikePlagiarism, scite, Recite, SciSpace) against a consistent set of criteria.
4. Specify functional and non-functional requirements with acceptance criteria.
5. Design the architecture: client, server with REST API, analysis service, storage.
6. Implement document upload and structure extraction, link and DOI checking (server).
7. Implement citation verification, summarisation and abstract comparison, requirement checks and topic similarity (analysis service).

8. Implement the user interface: upload, requirements window, report and visualisation (client).
9. Evaluate quality on a control sample and measure processing time.

1.4 Expected outcomes

Working web system (client, server, analysis service) with documentation; evaluation results on the control sample; recommendations for use at the School.

1.5 Engineering complexity

- Extraction of document structure (abstract, body, in-text citations, reference list) from DOCX/PDF in different citation styles (GOST, APA, IEEE).
- Link checking with redirects, DOI resolution, timeouts, paywalls, and soft-404 pages; matching retrieved metadata to the reference entry.
- Semantic comparison of a citation with source text, including paraphrase and cross-lingual cases (RU / KZ / EN).
- Summarisation of long documents and assessment of abstract–content consistency.
- Checking user-defined requirements (rule-based + intelligent) and similarity search over the topic archive.
- Integration of three independently developed components (JS client, Node.js server, Python service) through agreed APIs.

1.6 Scope

In scope: DOCX and text-based PDF documents in Russian, Kazakh and English; five checks (references, citations, abstract, user-defined requirements, topic similarity); roles student / supervisor / admin; web client. Out of scope: plagiarism detection (covered by existing tools), retrieval of paywalled full texts, grammar and style checking, mobile app (Could). Pilot: theses of the School of Software Engineering, 2022–2026.

1.7 Assumptions and constraints

- Full texts of sources are available only for open-access works; otherwise, the abstract/metadata is used, and the result is marked accordingly.
- The archive of past topics is provided by the School (dependency D1).
- Documents are confidential: they are not sent to external LLM services without the user’s consent (NFR-05).

2. Evidence of Relevance

Status of each claim: **V** – verified (source given), **A** – assumption (verification method given). **R** – rejected after verification.

2.1 Evidence collection plan

#	Method	Participant / source	Date	Status
S-1	Online survey (Google Forms, 9 questions, anonymous)	AITU students and academic staff; N = 26 (21 undergraduate, 1 master's, 2 PhD, 1 lecturer, 1 other)	23.10.2026	Done
I-1	Interview	Supervisor of diploma projects, School of SE	by 15.11.2026	Planned
I-2	Interview	Lecturer who reviews theses	by 15.11.2026	Planned
I-3	Interview	Norm-control / topic approval staff	by 15.11.2026	Planned
O-1	Observation	Supervisor checking a thesis reference list (time measured)	by 15.11.2026	Planned
D-1	Data analysis	10 past theses: share of broken links, references without DOI, citation style	by 20.12.2026 (with E1)	Planned
D-2	Data	Archive of past thesis topics from the School	24.10.2026	Requested
C-1	Agreement	Supervisor / School confirms scope and access to control sample	24.10.2026	Requested

Questionnaire: Appendix B.1; interview guide: Appendix B.2. Raw anonymised responses: docs/evidence/survey-S1.csv. The survey was conducted under the working name SciCheck.

2.2 Claims register

ID	Claim	Status	Evidence / verification
C 1	A notable share of web references in scholarly texts become broken or point to changed content	V (literature + survey)	Klein et al. 2014; Jones et al. 2016; S-1 Q6: 58% (15/26) have encountered broken or unverifiable references; Q5: 46% (12/26) name broken links / incorrect DOIs as the hardest part
C 2	Citations frequently misrepresent the cited source	V (literature + survey)	Jergas & Baethge 2015; Mogull 2017; S-1 Q5: 58% (15/26) name "checking whether sources support citations" as the hardest task
C 3	LLM-assisted writing increases fabricated references	V (literature + survey)	Walters & Wilder 2023; S-1 Q6: 58% (15/26) met references that could not be verified
C 4	Supervisors spend significant time checking references and citations manually	V (survey, partial)	S-1 Q4: 65% (17/26) spend ≥15 min per document, 27% (7/26) more than 30 min; Q3: 62% (16/26) open links one by one; supervisor time to be measured in O-1
C 5	Abstracts of student theses often do not reflect the content	V (survey)	S-1 Q5: 35% (9/26) find abstract accuracy hard to check; Q8: 35% (9/26) want abstract-vs-content comparison
C 6	Requirements differ by programme / journal, so a fixed checklist is insufficient	A (weak support)	S-1 Q5: 23% (6/26) find requirement compliance hard; Q8: 27% (7/26) want it — to be confirmed in I-3
C 7	Repeated or very similar topics occur across years and are detected manually	A (weak support)	S-1 Q5: 19% (5/26); Q8: 23% (6/26) want similar-topic detection — to be confirmed in I-3 and D-2
C 8	Plagiarism systems used at AITU do not check citation correctness	V (survey, indirect)	S-1 Q7: respondents rely on manual checking, DOI lookup or plagiarism tools; none of these verifies citation

ID	Claim	Status	Evidence / verification
			support
C 9	Users trust results only if the system shows evidence (source fragment, link status)	A	Not covered by S-1; to be checked in the usability test (E7) and interviews I-1…I-3

2.3 Requirements traceability

Requirements	Supported by
FR-10…FR-13 (references)	C1, C3, C4 (S-1)
FR-20…FR-22 (citations)	C2, C4, C8 (S-1: hardest task for 58%)
FR-30…FR-31 (abstract)	C5 (S-1)
FR-40…FR-43 (requirements window)	Supervisor’s task (justification letter); C6 weakly supported by S-1, to be confirmed in I-3
FR-50…FR-51 (topics)	Supervisor’s task (justification letter); C7 weakly supported by S-1, to be confirmed in I-3 and D-2

2.4 Findings summary

- Confirmed: C1/C3 — 58% have met broken or unverifiable references; C4 — 65% spend 15+ minutes per document, mostly opening links manually; C5 — 35% struggle to check abstract accuracy; C8 — checking whether sources support citations is the most frequently named difficulty (58%), and no tool currently in use addresses it.
- Weak / not confirmed: C6 (requirements compliance, 23–27%) and C7 (similar topics, 19–23%). These checks remain in scope because they are part of the supervisor’s task, but they are to be confirmed with norm-control staff (I-3) before pre-defense 2. C9 was not covered by the survey.
- New needs (Q9): 10 of 26 open answers were substantive: (1) accuracy and source reliability (-> FR-10…FR-22); (2) explained reports with recommendations for fixes (-> FR-65); (3) multilingual support and detection of AI-generated text (-> NFR-06, FR-64).
- Priorities (Q8): DOI and metadata verification 54%; link checking 42%; citation-support checking 42%; abstract-vs-content comparison 35%; requirements compliance 27%; similar topics 23%; issue report 8%.
- Limitations: small convenience sample (N = 26, one university, one day); 81% undergraduates, staff represented by one lecturer and two PhD students. Staff-specific claims (C4, C6, C7) will be confirmed by O-1 and I-1…I-3 before pre-defense 2.

3. Literature Review

Scope. Five streams that correspond to the five checks of the system: (1) reference rot and link integrity; (2) reference extraction and scholarly metadata; (3) citation accuracy and claim verification; (4) summarisation and abstract–content consistency; (5) text/topic similarity and AI support in academic review. **Totals:** 26 sources — 23 scientific publications (14 from 2020–2025) and 3 technical/normative documents. Foundational works before 2020 are kept where current methods build on them.

3.1 Critical review

Stream 1 — Reference rot (-> reference check)

Klein et al. [L1] analysed over 3.5 million scholarly articles and found that about one in five articles with web references suffers from *reference rot* (links that no longer resolve). Jones et al. [L2] showed that even links that still resolve often lead to changed content: about three out of four URI references pointed to content that had drifted from what was originally cited. **Implication:** checking only the HTTP status is insufficient — the system must also compare what the link leads to with the reference entry (title/author/year match), handle redirects, and prefer DOIs (FR-10, FR-11). Baseline in E2 = status-code-only check.

Stream 2 — Reference parsing and metadata (-> structure extraction, DOI resolution)

Tkaczyk et al. [L3] compared ten open-source reference parsers; GROBID performed best out of the box, and retraining on domain data improved all ML-based parsers. GOST-style Russian references are not represented in their benchmark — a limitation we must test (E1). Crossref [L4] is the authoritative registry of DOI metadata; OpenAlex [L5] and Semantic Scholar [L6] add open abstracts, open-access locations and multilingual coverage. Walters & Wilder [L7] found that a large share of bibliographic citations generated by ChatGPT were fabricated (55 % for GPT-3.5, 18 % for GPT-4), and many real ones contained errors — a growing risk for student texts. **Implication:** every reference is resolved against at least two sources; a reference that cannot be found anywhere is flagged as “possibly non-existent” (FR-12).

Stream 3 — Citation accuracy and verification (-> citation check)

A systematic review and meta-analysis by Jergas & Baethge [L8] estimated that about a quarter of quotations in medical journal articles contain errors, with a substantial share being major errors; Mogull [L9] showed that estimates depend strongly on study methodology. Automatic verification is studied as *scientific claim verification*: SciFact [L10] retrieves rationale sentences from abstracts and labels claims as SUPPORTS / REFUTES. Semantic matching relies on sentence embeddings: Sentence-BERT [L11], its multilingual distillation [L12], LaBSE [L13] and multilingual E5 [L14] allow cross-lingual comparison (RU/KZ citation vs. EN source). Limitations: source full text is often unavailable; Kazakh is under-represented in training data. **Implication:** two-stage approach — embedding retrieval of the most similar source fragment, then entailment (NLI) classification; labels *supported / partially supported / not supported / source unavailable*; TF-IDF cosine as the baseline (E3).

Stream 4 — Summarisation and abstract consistency (-> abstract check)

Long scientific documents require discourse-aware or long-context models [L15, L16]. Lexical metrics such as ROUGE [L17] correlate poorly with meaning; BERTScore [L18] compares texts in embedding space. For *consistency* between a summary and its source, NLI-based SummaC [L19] and AlignScore [L20] detect unsupported statements. **Implication:** the system (a) generates a reference summary of the body, (b) measures coverage of the body's key points by the author's abstract, and (c) flags abstract sentences not supported by the body; validated against expert ratings (E4).

Stream 5 — Similarity, topics and AI-assisted review (-> topic check, requirements check)

Foltýnek et al. [L21] reviewed academic plagiarism detection and distinguish lexical, syntactic and semantic methods; semantic (embedding-based) methods detect paraphrased ideas that lexical methods miss — the same applies to topic similarity across years. Liang et al. [L22] found LLM feedback on papers overlaps with human reviewers at a level comparable to human–human overlap, but is less specific; Kousha & Thelwall [L23] conclude AI suits supporting tasks such as screening and format checks. **Implication:** requirement checks are rule-based where measurable and LLM-assisted (with quoted evidence) only for qualitative items; all results are advisory.

Gap

Plagiarism tools detect copied text; citation tools check formatting or retractions. No reviewed tool verifies, in one workflow, link validity, citation content, abstract consistency, custom requirements and topic repetition — and none targets Russian/Kazakh student theses.

3.2 Analysis table

ID	Reference	Type	Method / content	Key finding	Limitation	Used for
L1	Klein M., Van de Sompel H., Sanderson R., et al. (2014). Scholarly context not found: One in five articles suffers from reference rot. *PLoS ONE*, 9(12), e115253. doi:10.1371/journal.pone.0115253	Journal	Large-scale link analysis	~1 in 5 articles affected by reference rot	STM corpora, 2014	C1; FR-10
L2	Jones S.M., Van de Sompel H., Shankar H., et al. (2016). Scholarly context adrift: Three out of four URI references lead to changed content. *PLoS ONE*, 11(12), e0167475. doi:10.1371/journal.pone.0167475	Journal	Content drift analysis	Working links often lead to changed content	Web archives needed	FR-11 metadata match; E2 baseline
L3	Tkaczyk D., Collins A., Sheridan P., Beel J. (2018). Machine learning vs. rules and out-of-the-box vs. retrained: An evaluation of open-source bibliographic reference and citation parsers. *JCDL '18*. arXiv:1802.01168	Conference	Benchmark of 10 parsers	GROBID best; retraining helps	No GOST/RU styles	ADR-004; E1

ID	Reference	Type	Method / content	Key finding	Limitation	Used for
L4	Hendricks G., Tkaczyk D., Lin J., Feeney P. (2020). Crossref: The sustainable source of community-owned scholarly metadata. *Quantitative Science Studies*, 1(1), 414–427. doi:10.1162/qss_a_00022	Journal	Metadata infrastructure	Authoritative open DOI metadata	Abstracts often missing	FR-11, FR-12
L5	Priem J., Piwowar H., Orr R. (2022). OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. arXiv:2205.01833	Preprint	Open scholarly graph	Free API, OA locations	Metadata gaps	Source text retrieval
L6	Kinney R., Anastasiades C., Authur R., et al. (2023). The Semantic Scholar Open Data Platform. arXiv:2301.10140	Preprint	Open data platform & API	Abstracts, OA PDFs, embeddings	Mostly English	Source text retrieval
L7	Walters W.H., Wilder E.I. (2023). Fabrication and errors in the bibliographic citations generated by ChatGPT. *Scientific Reports*, 13, 14045. doi:10.1038/s41598-023-41032-5	Journal	Evaluation of LLM-generated references	55 % (GPT-3.5) / 18 % (GPT-4) fabricated	Specific model versions	C3; FR-12
L8	Jergas H., Baethge C. (2015). Quotation accuracy in medical journal articles — a systematic review and meta-analysis. *PeerJ*, 3, e1364. doi:10.7717/peerj.1364	Journal (meta-analysis)	Meta-analysis of quotation errors	≈ 25 % of quotations erroneous	Medicine only	C2
L9	Mogull S.A. (2017). Accuracy of cited “facts” in medical research articles: A review of study methodology and recalculation of quotation error rate. *PLoS ONE*, 12(9), e0184727. doi:10.1371/journal.pone.0184727	Journal	Methodological review	Error rates depend on method	Medicine only	Annotation protocol E3
L10	Wadden D., Lin S., Lo K., et al. (2020). Fact or fiction: Verifying scientific claims. *EMNLP 2020*, 7534–7550. doi:10.18653/v1/2020.emnlp-main.609	Conference	Claim verification dataset & model	Rationale retrieval + labels feasible	Abstracts only, English	Citation-check pipeline
L11	Reimers N., Gurevych I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*, 3982–3992. doi:10.18653/v1/D19-1410	Conference	Sentence embeddings	Efficient semantic similarity	Monolingual	Similarity core
L12	Reimers N., Gurevych I. (2020). Making monolingual sentence embeddings multilingual using	Conference	Multilingual distillation	Cross-lingual similarity	KZ quality unknown	Model candidate (E3)

ID	Reference	Type	Method / content	Key finding	Limitation	Used for
	knowledge distillation. *EMNLP 2020*, 4512–4525. doi:10.18653/v1/2020.emnlp-main.365			incl. RU		
L13	Feng F., Yang Y., Cer D., Arivazhagan N., Wang W. (2022). Language-agnostic BERT sentence embedding. *ACL 2022*, 878–891. doi:10.18653/v1/2022.acl-long.62	Conference	109-language embeddings	Strong bitext retrieval incl. KZ	Less tuned for paraphrase	Model candidate (E3)
L14	Wang L., Yang N., Huang X., et al. (2024). Multilingual E5 text embeddings: A technical report. arXiv:2402.05672	Preprint	Multilingual retrieval embeddings	Strong multilingual retrieval	Large model sizes	Default model (ADR-005)
L15	Cohan A., Dernoncourt F., Kim D.S., et al. (2018). A discourse-aware attention model for abstractive summarization of long documents. *NAACL-HLT*, 615–621. doi:10.18653/v1/N18-2097	Conference	Long-document summarisation (arXiv/PubMed)	Section structure improves summaries	English	Summarisation design
L16	Beltagy I., Peters M.E., Cohan A. (2020). Longformer: The long-document transformer. arXiv:2004.05150	Preprint	Long-context transformer	Handles long scientific texts	Compute cost	Alternative (ADR-005)
L17	Lin C.-Y. (2004). ROUGE: A package for automatic evaluation of summaries. *Text Summarization Branches Out (ACL Workshop)*, 74–81	Workshop — foundational	n-gram overlap metric	Standard metric	Lexical only	Baseline metric E4
L18	Zhang T., Kishore V., Wu F., Weinberger K.Q., Artzi Y. (2020). BERTScore: Evaluating text generation with BERT. *ICLR 2020*. arXiv:1904.09675	Conference	Embedding-based metric	Better correlation with humans	Not a consistency metric	Abstract coverage score
L19	Laban P., Schnabel T., Bennett P.N., Hearst M.A. (2022). SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *TACL*, 10, 163–177. doi:10.1162/tacl_a_00453	Journal	Sentence-level NLI consistency	Detects unsupported summary sentences	English	Abstract consistency (FR-31)
L20	Zha Y., Yang Y., Li R., Hu Z. (2023). AlignScore: Evaluating factual consistency with a unified alignment function. *ACL 2023*. doi:10.18653/v1/2023.acl-long.634	Conference	Unified alignment model	State-of-the-art consistency scoring	English	Alternative scorer (E4)
L21	Foltýnek T., Meuschke N., Gipp B. (2019). Academic plagiarism	Journal (review)	Review of detection	Semantic methods	Pre-LLM era	Topic similarity

ID	Reference	Type	Method / content	Key finding	Limitation	Used for
	detection: A systematic literature review. *ACM Computing Surveys*, 52(6), 112. doi:10.1145/3345317		methods	catch paraphrase		(FR-50)
L22	Liang W., Zhang Y., Cao H., et al. (2024). Can large language models provide useful feedback on research papers? A large-scale empirical analysis. *NEJM AI*, 1(8). doi:10.1056/AIoa2400196	Journal	GPT-4 vs. human reviews	Useful but generic feedback	Paper-level only	Scope of LLM use
L23	Kousha K., Thelwall M. (2024). Artificial intelligence to support publishing and peer review: A summary and review. *Learned Publishing*, 37(1), 4–12. doi:10.1002/leap.1570	Journal (review)	Review of AI tools	AI suited to supporting checks	Fast-changing	Positioning as decision support
T1	GROBID documentation. https://grobid.readthedocs.io	Technical	PDF -> TEI structure	—	—	ADR-004
T2	Crossref REST API documentation. https://api.crossref.org	Technical	Works search, DOI metadata	—	Rate etiquette	FR-11, FR-12
T3	AITU diploma project formatting requirements / ГОСТ 7.1-2003 bibliographic description (School of Software Engineering, current academic year)	Normative	Requirements to check	—	—	FR-40 templates

4. Existing System Comparison

Systems (5): Turnitin, StrikePlagiarism, scite (Reference Check), Recite (ReciteWorks), SciSpace. **Criteria (9)** — the same for every system. Ratings: Yes full · Partly partial · No absent. Ratings were checked against vendor documentation on 23.10.2026.

#	Criterion	Linked requirement
K1	Link availability check (URL status, redirects)	FR-10
K2	DOI resolution and metadata match with the reference entry	FR-11, FR-12
K3	Citation content verification against source text	FR-20…22
K4	In-text citation <-> reference list consistency	FR-13
K5	Summary vs. author’s abstract	FR-30, FR-31
K6	User-defined requirements check	FR-40…43
K7	Topic similarity with a local archive of past theses	FR-50
K8	Russian / Kazakh language support	NFR-06
K9	Cost and data privacy (self-hosting)	NFR-05

Criterion	Turnitin	StrikePlagiarism	scite Reference Check	Recite	SciSpace	**Scirevia**
K1 Links	No	No	No *(not described in Reference Check documentation)*	No	No	Yes
K2 DOI / metadata	No	No	Yes checks references for retractions and editorial notices	No	No	Yes
K3 Citation content	No	No	Partly shows how others cite a paper (supporting / contrasting), not whether *your* citation matches	No	No	Yes
K4 Citation <-> list	No	No	No	Yes matches in-text citations with reference list (APA/Harvard)	No	Partly (Should)
K5 Abstract vs. content	No	No	No	No	Partly generates summaries, no comparison with abstract	Yes
K6 Custom requirements	No	No	No	Partly style rules only	No	Yes

Criterion	Turnitin	StrikePlagiarism	scite Reference Check	Recite	SciSpace	**Scirevia**
K7 Topic archive	Partly text similarity incl. institutional repository, not topic-level	Partly text similarity, institutional database *(incl. institutional archives, vendor information)*	No	No	No	Yes
K8 RU / KZ	Partly *(multilingual; Kazakh not documented)*	Yes *(multilingual; used by Kazakhstani universities)*	No mostly English corpus	No *(English-language styles only)*	Partly *(multilingual interface, mainly English corpus)*	Yes
K9 Cost / privacy	No institutional licence, cloud	No institutional licence, cloud	Partly freemium, cloud	Partly freemium, cloud	Partly freemium, cloud	Yes free, self-hosted at AITU

Conclusions

1. Plagiarism systems (Turnitin, StrikePlagiarism) answer *“is the text copied?”*, not *“are the references and citations correct?”*. Scirevia complements, not replaces them.
2. scite and Recite each cover one aspect of references (retractions; formatting consistency) and are oriented to English-language texts.
3. No tool covers K1, K3, K5, K6 and K7 together or supports RU/KZ theses — this justifies Scirevia.

Evidence (accessed 23.10.2026)

- scite: *How do I use the scite Reference Check?* — help.scite.ai; Nature Index news on scite’s retraction-flagging bot.
- Recite: reciteworks.com — product tour and FAQ.
- Turnitin: turnitin.com/products/similarity · StrikePlagiarism: strikeplagiarism.com · SciSpace: typeset.io
- Tools currently used by respondents (S-1, Q7): manual checking, DOI lookup and plagiarism-detection software; none of them verifies whether a source supports the citation.

5. Technology Comparison

Scores 1–3 (3 = best) against criteria derived from requirements. Final decisions: adr/.

D1. Server platform

Requirements: concurrent link checking with timeouts (FR-10, NFR-02), file upload (FR-01), REST API with documentation (NFR-07), team skills of the backend developer (JavaScript, REST, NoSQL).

Criterion	**Node.js + Express**	Python + FastAPI	NestJS (TypeScript)
Asynchronous I/O for hundreds of HTTP checks	3	3	3
Team expertise (backend developer)	3	2	1
Simplicity / learning curve	3	3	1
Upload, validation, Swagger ecosystem	3 (multer, joi, swagger-jsdoc)	3	3
Access to ML libraries	1 — not needed: ML is in a separate service (D3)	3	1
Total	**13**	14	9

Decision: Node.js + Express. FastAPI scores one point higher only because of ML libraries, which the server does not need (they are isolated in the analysis service); team expertise outweighs this. -> ADR-001

D2. Data storage

Requirements: documents and nested check reports of variable structure (FR-60), requirement templates (FR-43), topic archive (FR-51), history (FR-62).

Criterion	**MongoDB**	PostgreSQL	PostgreSQL + JSONB
Nested, evolving report structure	3	1	3
Team expertise	3	2	2
File storage (GridFS) or references	3	2	2
Relational integrity (users–documents–reports)	2	3	3
Vector search for topics	2 (Atlas only; we keep vectors in analysis service)	2 (pgvector)	2 (pgvector)
Total	**13**	10	12

Decision: MongoDB with Mongoose schemas for validation; topic embeddings are stored and searched inside the analysis service (corpus of a few thousand topics fits in memory / FAISS). -> ADR-002

D3. Interaction between components

Requirements: long checks (citation verification may take minutes, NFR-02), progress visible to user (FR-63), independent development of three components.

Criterion	Synchronous REST	**REST + asynchronous jobs (job record in MongoDB, status polling, callback from analysis service)**	Message broker (RabbitMQ / BullMQ + Redis)
No timeouts on long checks	1	3	3
Progress per check	1	3	3
Extra infrastructure	3	3	1
Complexity for the team	3	2	1
Total	8	**11**	8

Decision: REST/JSON between all components; checks run as asynchronous jobs. Upgrade path: BullMQ if load grows. API contracts are fixed in OpenAPI before implementation. -> ADR-003

D4. Document structure extraction

Requirements: abstract, body, in-text citations, reference list from DOCX and PDF; GOST/APA/IEEE styles (FR-02).

Criterion	**python-docx / PyMuPDF + rules**	**GROBID**	Apache Tika
Reference list parsing into fields	1	3	1
DOCX support	3	1 (PDF only)	3
GOST / Russian references	2 (custom rules)	1–2 *(test in E1)*	1
Deployment	3	2 (Java service)	2

Decision: hybrid — python-docx for DOCX (most theses), GROBID for PDF and for parsing reference strings; rule-based fallback for GOST. Chosen by E1 F1-score. -> ADR-004

D5. Semantic comparison (citations, topics)

Criterion	TF-IDF / BM25	**Multilingual sentence embeddings (multilingual-e5 / LaBSE / paraphrase-multilingual)**	LLM API only
Detects paraphrase	1	3	3
Cross-lingual RU/KZ <-> EN	1	3	3
Cost / runs locally	3	3	1
Explainability (fragment + score)	3	2	2

Decision: embeddings for retrieval of the most similar source fragment + NLI cross-encoder for the label; TF-IDF kept as the **baseline** in E3/E5. LLM optional for borderline cases (with consent). -> ADR-005

D6. Summarisation and abstract comparison

Criterion	Extractive (TextRank)	Local abstractive model (mT5 / mBART)	**LLM API (with consent) + extractive fallback**
Summary quality on long RU/KZ text	1	2	3
Cost / hardware	3	1 (GPU)	2

Criterion	Extractive (TextRank)	Local abstractive model (mT5 / mBART)	**LLM API (with consent) + extractive fallback**
Privacy	3	3	1 -> mitigated by consent / local fallback

Decision: LLM summary when the user consents; otherwise extractive summary. Abstract comparison uses embedding coverage + NLI consistency (independent of summariser).

D7. Client

Criterion	**HTML/CSS/JavaScript + Chart.js**	Reac t	Flutter (web + mobile)
Team expertise (frontend developer)	3	2	3
Complexity of report UI (tabs, tables, side-by-side)	2	3	2
Build tooling / simplicity	3	2	2
Mobile version	1	1	3

Decision: HTML/CSS/JS (ES modules) for the web client; Flutter mobile version is optional (Could).

6. Requirements Specification

Priority: **M** must · **S** should · **C** could. Every M requirement has an acceptance criterion (AC) verifiable by a test or experiment in the verification plan. Owner: **SS** — Sergeyeva (analysis), **AK** — Kuanyshbekova (client), **MK** — Khairgeldinova (server).

6.1 Functional requirements

Document upload and structure

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-01	Upload DOCX or text PDF (<= 20 MB) and select which checks to run	M	Valid files accepted; other types rejected with message; test with 10 files	MK, AK
FR-02	Extract title, abstract, body, in-text citations and reference list	M	On control sample (E1): reference entries F1 >= 0.85, abstract found in >= 18/20 documents	MK, SS
FR-03	Show extracted structure to the user and allow correcting abstract / reference boundaries	S	User edit is used in subsequent checks (E2E test)	AK

Reference check

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-10	For each URL: status (working / broken / redirected / access restricted / timeout), final URL, HTTP code	M	On 200 labelled links (E2): accuracy >= 0.90; timeout per link <= 10 s	MK
FR-11	Resolve DOI and compare retrieved metadata (title, authors, year) with the reference entry; match score	M	On 100 labelled references: correct “matches / does not match” >= 0.90	MK, SS
FR-12	For references without DOI/URL, search Crossref / OpenAlex / Semantic Scholar; flag “not found in any source”	M	Recall of planted fabricated references >= 0.8 (E2b)	SS
FR-13	Check consistency of in-text citations and reference list (unused entries, missing entries)	S	Synthetic test with 5 planted inconsistencies — all found	SS

Citation check

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-20	For each citation, retrieve source text (OA full text or abstract)	M	Retrieval attempt logged per citation; status “source unavailable” when not found	SS
FR-21	Find the most similar source fragment and assign label: supported / partially supported / not supported / source unavailable, with score	M	E3: macro-F1 >= 0.70 and >= +0.10 over TF-IDF baseline	SS
FR-22	Display citation and source fragment side by side with link to source	M	UI test: every label has fragment or reason	AK

Abstract check

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-30	Generate a summary of the document body	M	Summary produced for 20/20 control documents <= 2 min each	SS
FR-31	Compare author's abstract with content: coverage score, key points missing from the abstract, abstract sentences not supported by the body	M	E4: Spearman ρ >= 0.6 between system score and expert rating	SS

Requirements window

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-40	User enters requirements in a separate window: free text or structured items (sections, volume, number and age of references, style, keywords, abstract length)	M	Requirements saved with the check; UI test	AK, MK
FR-41	Measurable requirements are checked by rules	M	10 rule types; unit tests per rule; precision = recall = 1.0 on synthetic documents	SS
FR-42	Qualitative requirements (e.g. "aim and objectives are stated") are checked intelligently and each verdict includes a quote from the document	S	100 % of verdicts contain a quote existing verbatim in the document	SS
FR-43	Save and reuse requirement templates (e.g. "AITU diploma project")	S	Template applied to a new check without re-entry	MK, AK

Topic check

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-50	Compare topic (title + abstract) with archive of past topics; show top-5 similar with score, year	M	E5: Precision@5 >= 0.6, MRR >= baseline + 0.1	SS
FR-51	Admin uploads / updates archive (CSV / XLSX)	M	1 000-row file imported <= 1 min; index rebuilt	MK

Report, users, history

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-60	Single report: summary panel + section per check with statuses	M	All selected checks present; matches API data (E2E)	AK
FR-61	Export report to PDF	S	PDF opens; content equals on-screen report	MK
FR-62	History of checks per user; compare two versions of the same document	C	—	MK, AK
FR-64	Flag passages that are possibly AI-generated for the reviewer (from S-1, Q9)	C	—	SS
FR-65	For each detected issue, show a short recommendation on how to fix it (from S-1, Q9)	C	—	AK, SS
FR-63	Progress of running checks	M	Status updates at least every 5 s	MK, AK

ID	Requirement	Pr.	Acceptance criterion	Owner
FR-70	Roles: student, supervisor, admin; registration/login with AITU e-mail	M	Access matrix tests: forbidden actions -> 403	MK
FR-71	Supervisor sees reports of documents shared by students	S	Sharing test	MK, AK

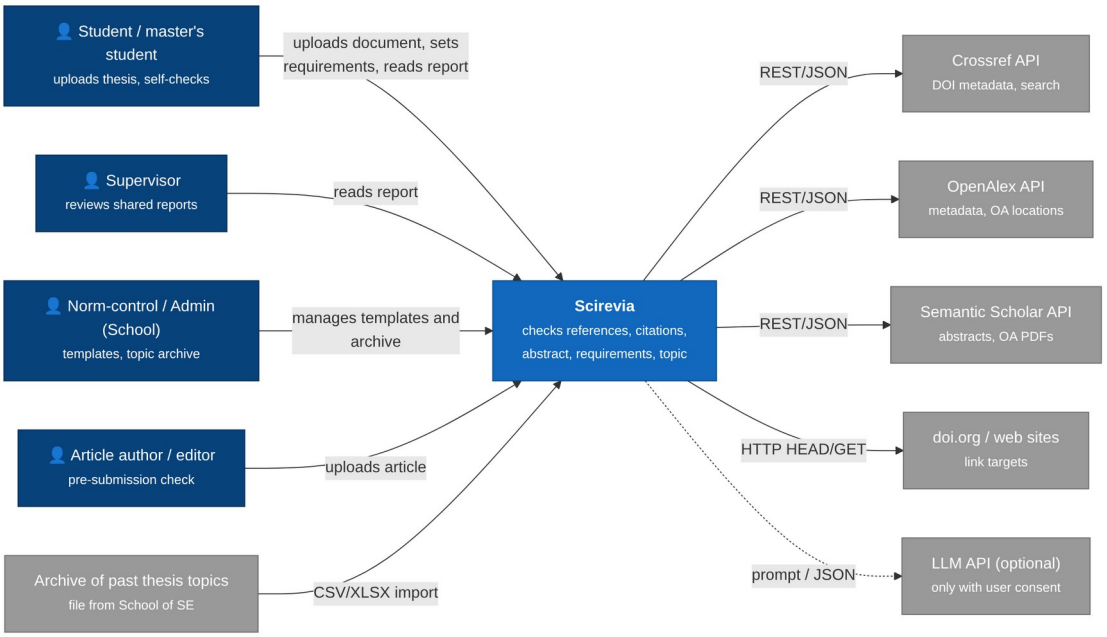
6.2 Non-functional requirements

ID	Requirement	Pr.	Acceptance criterion
NFR-01	Usability	M	SUS >= 70 with 5 users (E7)
NFR-02	Performance: full check of 80-page thesis with <= 60 references	M	<= 5 min on target server (E6)
NFR-03	Politeness to external services	M	Concurrency <= 10 per host; User-Agent with contact; Crossref/OpenAlex mailto set
NFR-04	Reliability	S	Failure of one check does not fail the report (fault-injection test)
NFR-05	Privacy	M	Documents not sent to external LLM without consent (network test); files deleted after 30 days (configurable)
NFR-06	Languages	M	Documents in RU, KZ, EN processed; UI in RU/EN (KZ — Should)
NFR-07	API documentation	M	OpenAPI for server and analysis service; contract tests pass
NFR-08	Maintainability	S	Test coverage >= 60 % per component; CI on every PR
NFR-09	Security	M	Passwords hashed (bcrypt), JWT, input validation, upload type checks; no secrets in Git

7. Architecture and Diagrams

Sources: diagrams/src/ (Mermaid .mmd, PlantUML .puml) · exports: diagrams/export/.

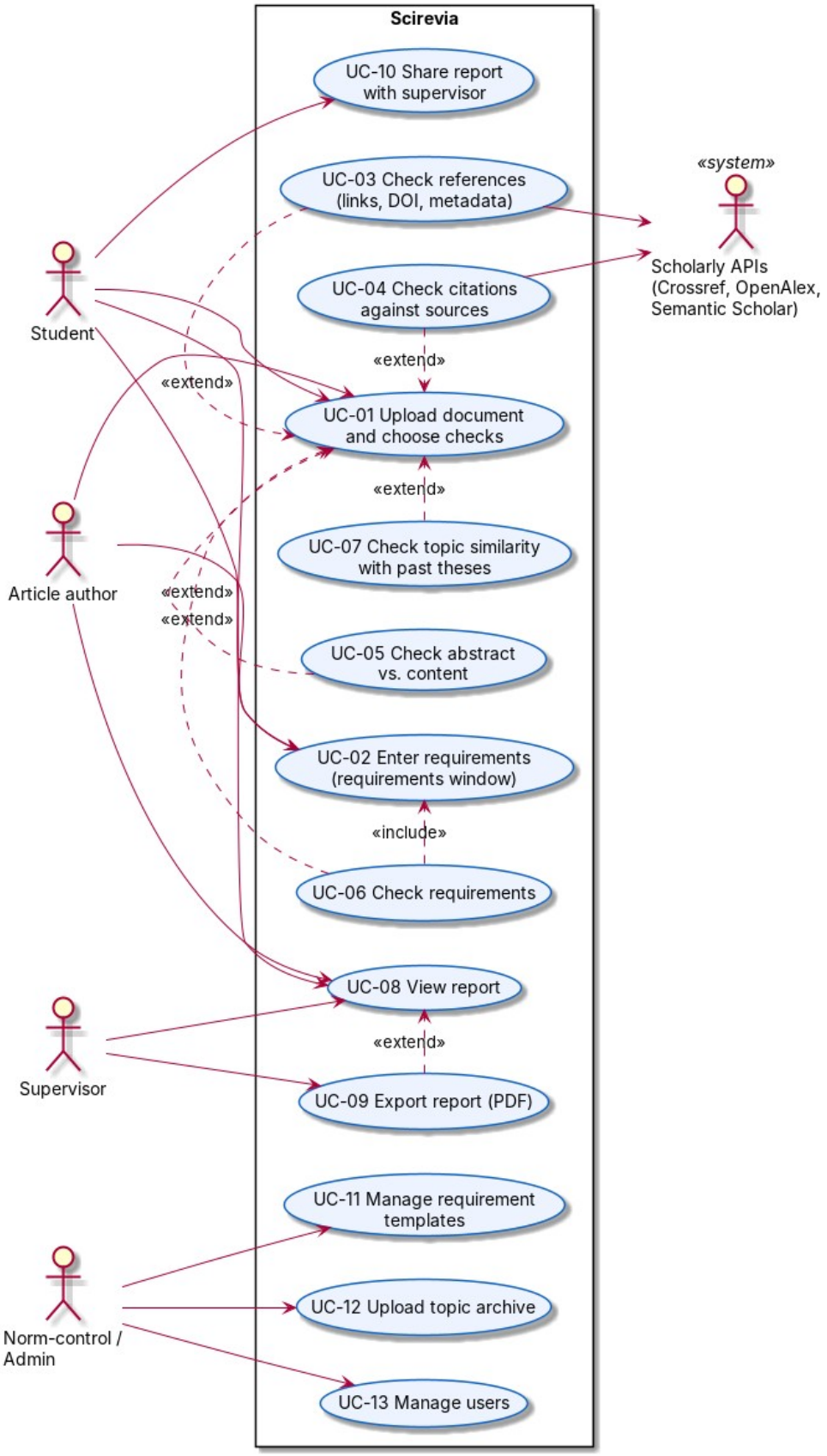
7.1 System context



context

Scirevia is used by students, article authors, supervisors and norm-control/admin staff of the School. It calls Crossref, OpenAlex and Semantic Scholar for metadata and source texts, performs HTTP requests to link targets and doi.org, imports the archive of past thesis topics provided by the School, and calls an LLM API **only** when the user consents (NFR-05).

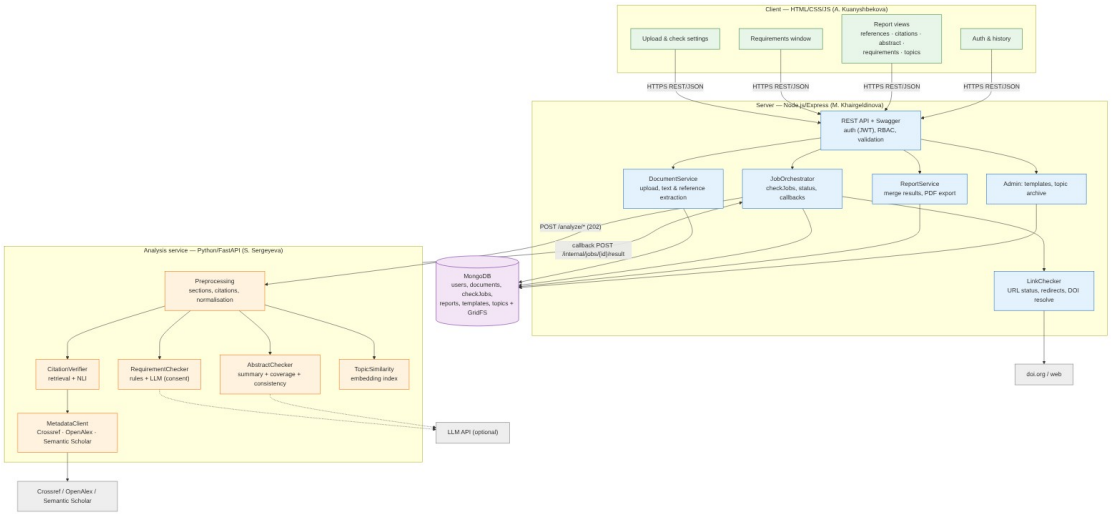
7.2 Use cases


Scirevia
UC-10 Share report with supervisor
UC-03 Check references (links, DOI, metadata)
UC-04 Check citations against sources
UC-01 Upload document and choose checks
UC-07 Check topic similarity with past theses
UC-05 Check abstract vs. content
UC-02 Enter requirements (requirements window)
UC-06 Check requirements
UC-08 View report
UC-09 Export report (PDF)
UC-11 Manage requirement templates
UC-12 Upload topic archive
UC-13 Manage users
«system»
Scholarly APIs (Crossref, OpenAlex, Semantic Scholar)
Student
Article author
Supervisor
Norm-control / Admin
«extend»
«extend»
«extend»
«extend»
«extend»
«extend»
«include»

use cases

Use case	Requirements	Prototype screen	Owner
UC-01 Upload document, choose checks	FR-01, FR-02, FR-03	New check	MK, AK
UC-02 Enter requirements	FR-40, FR-43	New check -> Requirements window	AK
UC-03 Check references	FR-10…13	Report -> References	MK, SS
UC-04 Check citations	FR-20…22	Report -> Citations	SS
UC-05 Check abstract	FR-30, FR-31	Report -> Abstract	SS
UC-06 Check requirements	FR-41, FR-42	Report -> Requirements	SS
UC-07 Check topic similarity	FR-50	Report -> Topic	SS
UC-08 View report	FR-60, FR-63	Report -> Overview	AK
UC-09 Export PDF	FR-61	Report -> Export	MK
UC-10 Share with supervisor	FR-71	Report -> Share	MK, AK
UC-11 Templates · UC-12 Topic archive · UC-13 Users	FR-43, FR-51, FR-70	Admin	MK

7.3 Components (draft)



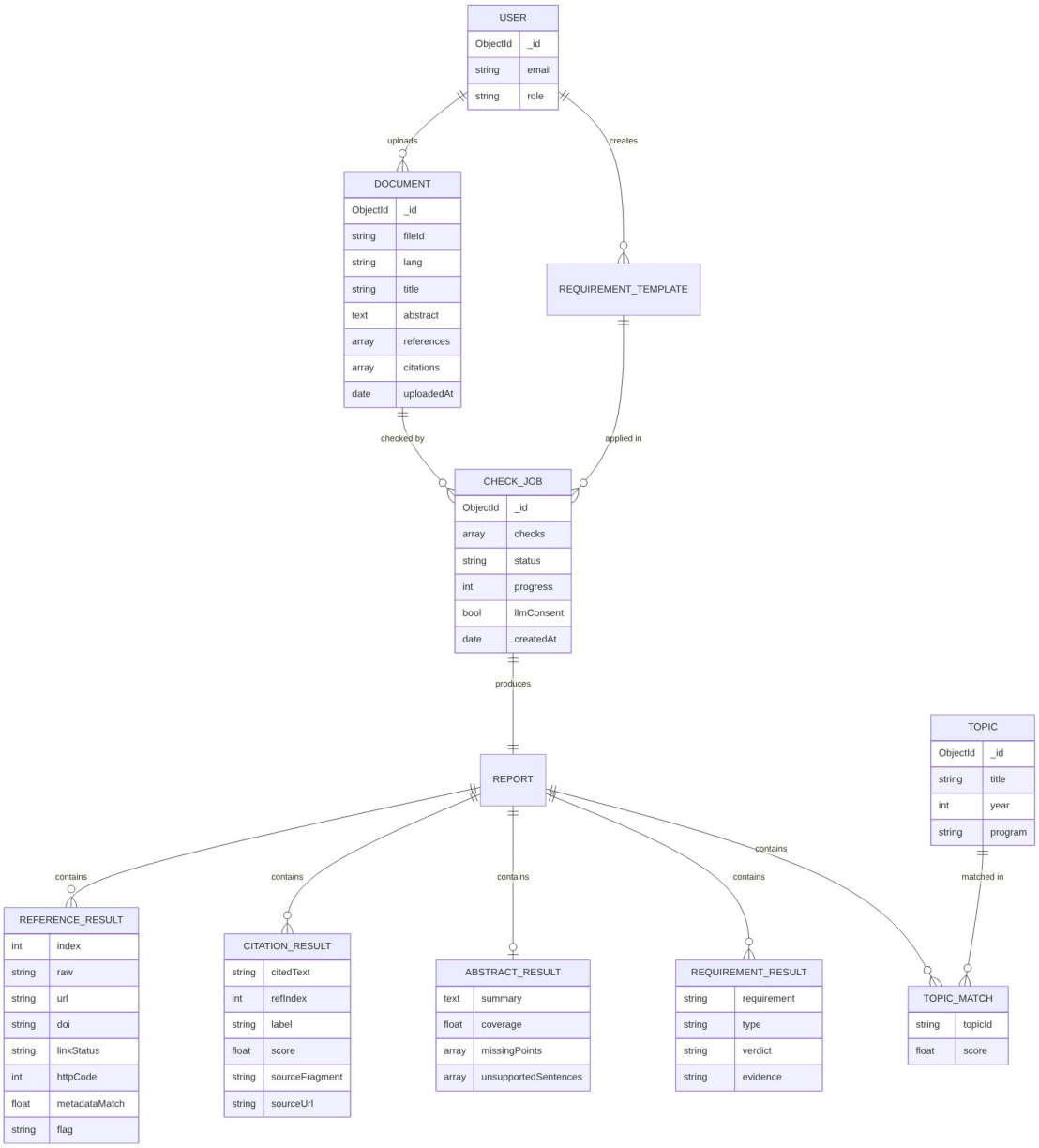
components

Component	Responsibility	Technology	Owner
Client	Upload, requirements window, report views, charts, history	HTML/CSS/JS, Chart.js	A. Kuanyshbekova
REST API	Auth (JWT), roles, validation, Swagger	Express, joi, swagger-jsdoc	M. Khairgeldinova
DocumentService	Upload, text extraction, reference list extraction	multer, mammoth / call to analysis preprocessing	M. Khairgeldinova
LinkChecker	URL status, redirects, timeouts, DOI resolution	undici/axios, p-limit	M. Khairgeldinova
JobOrchestrator	checkJobs, progress, callbacks, retries	Express, MongoDB	M. Khairgeldinova
ReportService	Merge results, PDF export	Puppeteer / pdfkit	M.

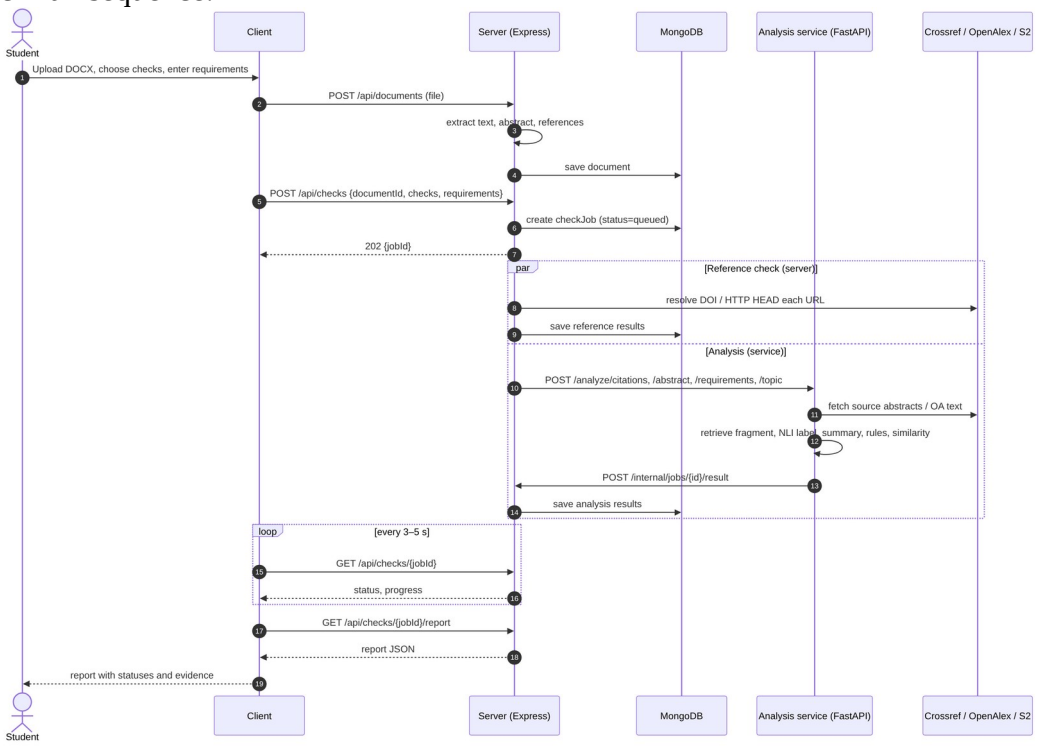
Component	Responsibility	Technology	Owner
			Khairgeldinova
Preprocessing	Sections, in-text citations, reference parsing, normalisation	spaCy/NLTK, GROBID	S. Sergeyeva
MetadataClient	Crossref, OpenAlex, Semantic Scholar	httpx	S. Sergeyeva
CitationVerifier	Retrieval of source fragment + NLI label	sentence-transformers, Transformers	S. Sergeyeva
AbstractChecker	Summary, coverage, consistency	Transformers / LLM API	S. Sergeyeva
RequirementChecker	Rule checks + LLM checks with quotes	Python, LLM API	S. Sergeyeva
TopicSimilarity	Embedding index of past topics	sentence-transformers, FAISS / NumPy	S. Sergeyeva
MongoDB	users, documents, checkJobs, reports, templates, topics, GridFS	MongoDB 7	M. Khairgeldinova

7.4 Additional views

- Data model:



- Check run sequence:



7.5 Interfaces between team members

Interface	Provider -> Consumer	Contract file	Task
Client <-> Server	MK -> AK	docs/api/openapi-server.yaml	SCI-9
Server <-> Analysis	SS -> MK	docs/api/openapi-analysis.yaml	SCI-9

7.6 Consistency check

Every M requirement -> >= 1 use case (7.2) -> >= 1 component (7.3) -> >= 1 test/experiment (Section 9.2–9.3). Re-check after each requirements change.

8. UX / Prototype

Clickable prototype in Figma (English version, 7 screens, fictional data) — link in the Project Card. Screenshots: prototype/screens/. Figma design: A. Kuanyshbekova.

Screen	Scenario	Use cases
New check	Upload -> choose checks -> requirements window (template + free text) -> consent for external AI -> start	UC-01, UC-02
Report overview	Summary of five checks -> export / share	UC-08…10
References	Status of each link / DOI, metadata match, possibly non-existent references	UC-03
Citations	Citation and closest source fragment side by side, label and score	UC-04
Abstract	Author’s abstract vs. generated summary, coverage, unsupported sentences	UC-05
Requirements	Rule and intelligent checks with evidence	UC-06
Topic	Top similar topics from the archive	UC-07

API examples (contract draft)

```
POST /api/checks
{ "documentId": "6531…", "checks":
["references","citations","abstract","requirements","topic"],
  "templateId": "aitu-diploma", "customRequirements": ["Each objective
has a result in the conclusion"],
  "llmConsent": false }
→ 202 { "jobId": "j_812", "status": "queued" }

GET /api/checks/j_812
→ { "status": "running", "progress": { "references": "done", "citations":
"running", "abstract": "queued", ... } }

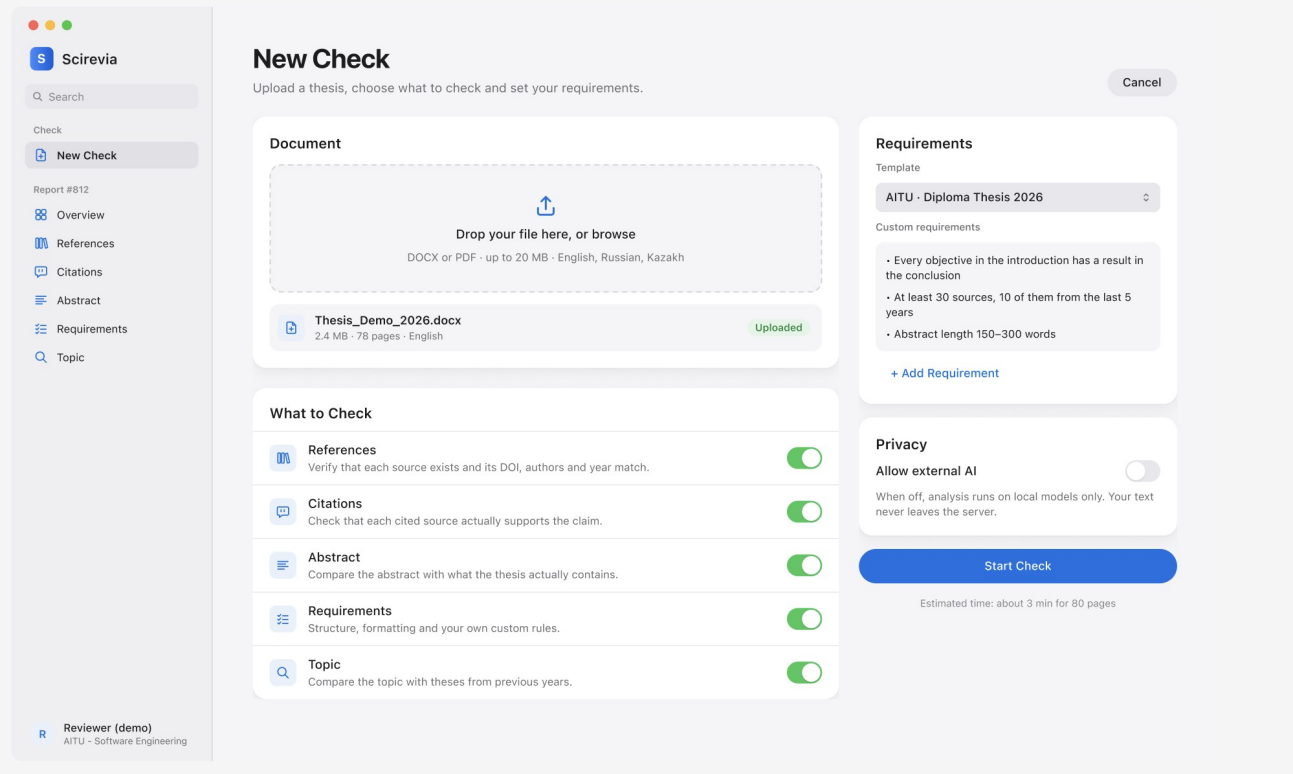
POST /analyze/citations           (server → analysis service)
{ "jobId": "j_812", "lang": "ru",
  "citations": [{ "id": "c14", "text": "Sentence-BERT позволяет …",
"refIndex": 3 }],
  "references": [{ "index": 3, "doi": "10.18653/v1/D19-1410" }] }
→ 202

callback  POST /internal/jobs/j_812/result
{ "check": "citations", "results": [
   { "id": "c14", "label": "supported", "score": 0.87,
     "sourceFragment": "…derive semantically meaningful sentence
embeddings…",
     "sourceType": "abstract", "sourceUrl":
"https://doi.org/10.18653/v1/D19-1410" } ] }
```

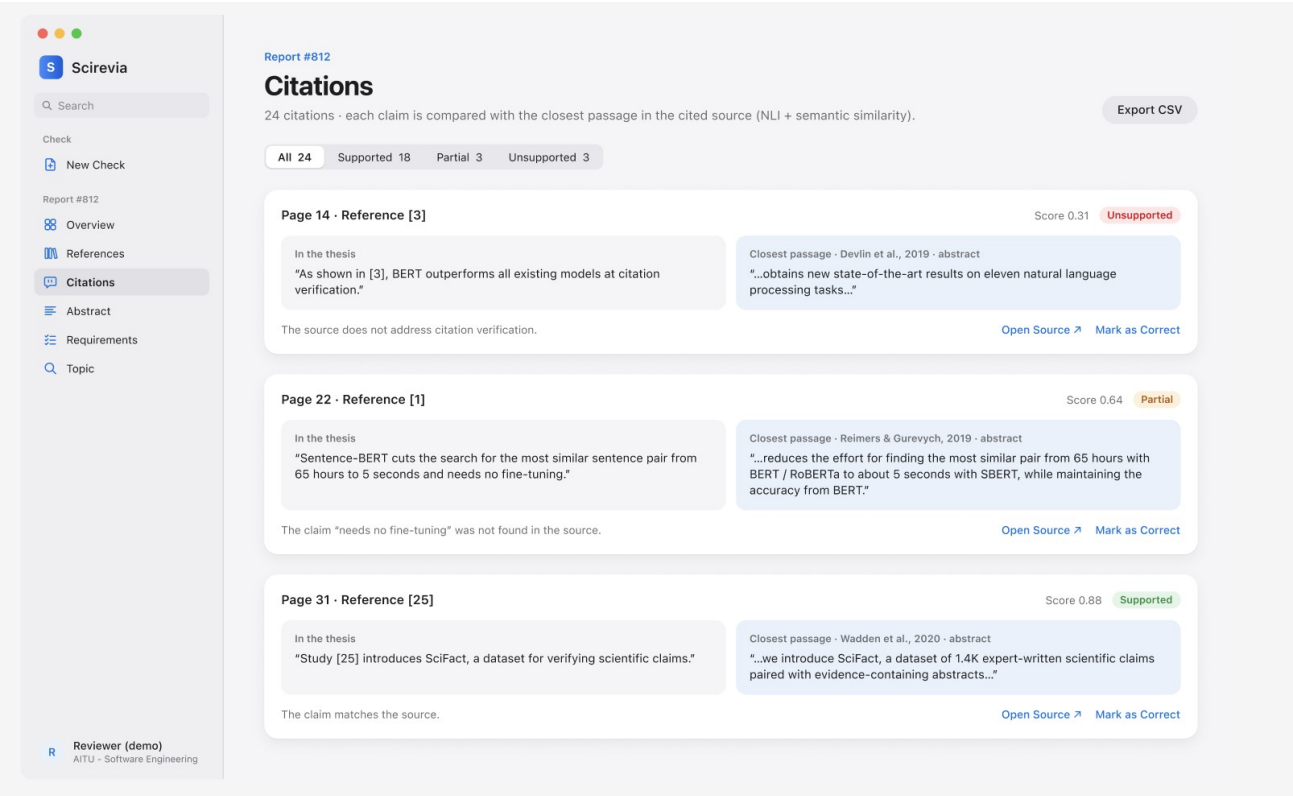
Usability test

5 participants (2 students, 2 supervisors, 1 norm-control), tasks: run a check with a template; find a broken reference; explain why a citation is “not supported”; add a custom requirement; export the report. Think-aloud + SUS. Results -> docs/evidence/usability-prototype.md.

Prototype screens



01-new



04-cites

9. Verification and Evaluation Plan

9.1 How the aim is demonstrated

The aim is achieved if, on a control sample of real theses, Scirevia (a) detects reference, citation, abstract, requirement and topic problems with accuracy above the defined thresholds and above simple baselines, (b) processes a thesis within the time limit, and (c) is usable and saves reviewers' time compared with manual checking.

9.2 Testing levels

Level	Client (AK)	Server (MK)	Analysis service (SS)
Unit	Jest for UI logic	Jest: validation, link classifier, job state machine	pytest: parsers, rules, scoring
Integration	—	supertest + MongoDB test container; mocked analysis service	recorded API responses (Crossref/OpenAlex)
Contract	—	OpenAPI validation of requests/responses (both directions)	same
End-to-end	Playwright: 5 key scenarios from prototype	<-	<-
Performance	—	10 concurrent checks (k6)	profiling per check
Security / privacy	—	OWASP ZAP baseline; auth matrix	network capture: no outbound text when llmConsent=false

CI: GitHub Actions on every pull request (NFR-08).

9.3 Control sample

- **Documents:** 20 theses of the School of SE (2022–2026, anonymised, with permission) + 5 published articles. *(dependency: School / supervisor)*
- **Topic archive:** list of past topics from the School (D-2).
- **Annotation:** two team members annotate independently, disagreements resolved with the supervisor; agreement reported as Cohen's κ.

9.4 Experiments, metrics, baselines

#	What is measured	Ground truth	Metric	Baseline	Target
E1	Structure extraction (FR-02)	Manually marked abstract, references, in-text citations in 20 docs	Precision / recall / F1 of reference entries; abstract found	GROBID only vs. rules only	F1 >= 0.85
E2	Link & DOI check (FR-10, FR-11)	200 links + 100 references labelled manually (working / broken / redirected-changed / wrong DOI)	Accuracy, per-class F1	HTTP-status-only check	Accuracy >= 0.90 and > baseline
E2 b	Non-existent references (FR-12)	20 planted fabricated references + 80 real	Recall, precision	—	Recall >= 0.8

#	What is measured	Ground truth	Metric	Baseline	Target
E3	Citation verification (FR-21)	150 citation–source pairs, labels: supported / partial / not supported	Macro-F1, confusion matrix; separately for RU, KZ, EN	TF-IDF cosine with tuned threshold	Macro-F1 >= 0.70 and >= baseline + 0.10
E4	Abstract assessment (FR-31)	2 experts rate abstract adequacy 1–5 for 20 theses	Spearman ρ between system score and mean expert rating	ROUGE-L between abstract and summary	ρ >= 0.6 and > baseline
E5	Topic similarity (FR-50)	30 query topics with labelled similar topics in archive	Precision@5, MRR	Jaccard similarity of keywords	P@5 >= 0.6; MRR >= baseline + 0.1
E6	Processing time (NFR-02)	20 docs, target server spec recorded	Median and max time per document	—	<= 5 min
E7	Usability and time saved (NFR-01)	5 users; reviewer checks references/citations of one thesis manually vs. with Scirevia	SUS; time on task; problems found	Manual check	SUS >= 70; >= 40 % time reduction

9.5 Conditions and threats to validity

Model versions and API snapshot dates are recorded; experiments repeated 3 times for LLM-based parts (temperature 0). Small sample from one School; annotators are team members (mitigated by supervisor adjudication and κ); limited open full texts -> “source unavailable” share is reported separately rather than counted as errors.

10. Project Management and Risks

- **YouTrack:** project key SCI — https://sergeyevasss.youtrack.cloud/projects/SCI; backlog import: youtrack/tasks.csv.
- **Git: main = reviewed baseline; branch SCI-<n>-<short-name>; commit message SCI-<n>: <description>; every PR is reviewed by another team member; the supervisor reviews significant changes at meetings. Tags: pre-defense-1, pre-defense-2, final-pre-defense.**
- **Meetings with supervisor:** twice a month (online/offline); each member reports on her part; decisions logged in Section 11.

10.1 Responsibilities

Area	S. Sergeyeva (ML/Data)	A. Kuanyshbekova (Frontend, UX/UI)	M. Khairgeldinova (Backend)	Supervisor
Requirements, interviews	R	R (scenarios)	C	A
Literature review	R	C	C	A
System & technology comparison	R (NLP)	R (client)	R (server, DB)	A
Architecture, API contracts	R (analysis API)	C	R (server API)	A
Prototype / Figma	C	R	C	I
Analysis service	R	I	C	C
Server, link check, storage	C	I	R	C
Client	I	R	C	I
Integration & testing, evaluation	R (quality)	R (usability)	R (E2E, perf.)	A

R — responsible · A — approves · C — consulted · I — informed.

10.2 Milestones (aligned with the supervisor's plan)

#	Stage	Result	Date
M1	Requirements and data analysis	Scenarios, survey S-1 (N = 26), request for topic archive and control sample; pre-defense 1 (tag pre-defense-1)	24.10.2026
M2	Design	Architecture, API specification, storage schema, UI mockups (Figma)	15.11.2026
M3	Base version	Upload, reference list extraction, link checking (E1, E2)	20.12.2026
M4	Text analysis	Citation check, summary and abstract comparison (E3, E4)	15.02.2027
M5	Requirements & topics	Requirements window, requirement checks, topic similarity (E5); **pre-defense 2**	March 2027 (School schedule)
M6	Integration & testing	Integrated system, evaluation on control sample (E6, E7)	15.04.2027
M7	Documentation	Explanatory note, presentation; **final pre-defense**	May 2027 (School schedule)

10.3 Risks

ID	Risk	P	I	Mitigation	Owner
R 1	Topic archive / control sample not provided in time by the School	M	H	Request at M1 via supervisor; fallback — topics from public AITU repository / own collection	SS
R 2	Open full texts unavailable for many sources	H	M	Use abstracts; explicit “source unavailable”; report share separately	SS
R 3	Low quality for Kazakh texts	M	M	Compare 3 multilingual models in E3; report per language	SS
R 4	GOST references parsed poorly	M	M	Rule-based fallback; retrain/adjust on 20 annotated docs	SS, MK
R 5	Integration problems between JS and Python components	M	H	OpenAPI contracts first (SCI-9); mocks; weekly integration build	all
R 6	Websites block automated link checks (403/429)	M	L	Browser-like UA, retries, class “access restricted” instead of “broken”	MK
R 7	Schedule slip in one component	M	H	Bi-weekly control; Must/Should priorities; reserve time before defence	supervisor
R 8	Confidentiality of student theses	L	H	Local processing by default, consent for LLM, retention 30 days, no theses in Git	all
R 9	LLM cost / API access	M	M	LLM only optional; local models for core checks	SS

10.4 Dependencies

- D1: archive of past topics and control sample from School of SE (blocks FR-50, E1–E5).
- D2: agreed API contracts (blocks parallel implementation).
- D3: server for deployment and model inference (to be requested from AITU IT at M2).
- D4: AITU official formatting requirements document (blocks template FR-43).

YouTrack backlog (initial)

ID	Task	Assignee	Priority	Due
SCI-1	Create repository, README, structure, .gitignore	Meruert Khairgeldinova	Critical	2026-10-20
SCI-2	Project proposal and scope	Saltanat Sergeyeva	Critical	2026-10-21
SCI-3	Survey questionnaire (S-1) and interview guide	Alzira Kuanyshbekova	Critical	2026-10-21
SCI-4	Survey S-1 (Google Forms, N = 26); update claims register	Alzira Kuanyshbekova	Critical	2026-10-23
SCI-5	Literature review (>=12 sources, >=8 scientific) + table	Saltanat Sergeyeva	Critical	2026-10-22
SCI-6	Existing systems comparison (5 systems, 9 criteria)	Alzira Kuanyshbekova	Critical	2026-10-22
SCI-7	Technology comparison + ADR-001…005	Meruert Khairgeldinova	Critical	2026-10-22
SCI-8	Requirements v0.1 with acceptance criteria	Saltanat Sergeyeva	Critical	2026-10-23
SCI-9	API contracts: openapi-server.yaml, openapi-analysis.yaml	Meruert Khairgeldinova	Critical	2026-10-30

ID	Task	Assignee	Priority	Due
SCI-10	Diagrams: context, use case, component (sources + exports)	Meruert Khairgeldinova	Critical	2026-10-23
SCI-11	Data model and sequence diagram	Meruert Khairgeldinova	Normal	2026-10-23
SCI-12	Clickable prototype + Figma mockups	Alzira Kuanyshbekova	Critical	2026-10-23
SCI-13	Verification and evaluation plan	Saltanat Sergeyeva	Critical	2026-10-23
SCI-14	Project plan, RACI, risks; supervisor log	Alzira Kuanyshbekova	Major	2026-10-23
SCI-15	Request topic archive and control sample from School	Saltanat Sergeyeva	Critical	2026-10-24
SCI-16	Spike: reference parsing, DOI extraction, TF-IDF citation baseline	Saltanat Sergeyeva	Major	2026-10-24
SCI-17	Tag pre-defense-1 and submission summary	Meruert Khairgeldinova	Critical	2026-10-24
SCI-18	Server skeleton: Express, Mongoose, auth, Swagger, Docker Compose	Meruert Khairgeldinova	Critical	2026-11-15
SCI-19	Document upload and text extraction (FR-01, FR-02)	Meruert Khairgeldinova	Critical	2026-12-01
SCI-20	Link checker and DOI resolution (FR-10, FR-11)	Meruert Khairgeldinova	Critical	2026-12-20
SCI-21	Annotate control sample (references, citations, abstracts)	Saltanat Sergeyeva	Critical	2026-12-10
SCI-22	Reference parsing incl. GOST; experiment E1	Saltanat Sergeyeva	Critical	2026-12-20
SCI-23	Metadata client + non-existent reference detection (FR-12)	Saltanat Sergeyeva	Critical	2027-01-20
SCI-24	Citation verification: embeddings + NLI; experiment E3	Saltanat Sergeyeva	Critical	2027-02-15
SCI-25	Summary and abstract comparison; experiment E4	Saltanat Sergeyeva	Critical	2027-02-28
SCI-26	Client: upload, check settings, progress	Alzira Kuanyshbekova	Critical	2026-12-20
SCI-27	Client: requirements window + templates	Alzira Kuanyshbekova	Critical	2027-02-15
SCI-28	Client: report views and charts	Alzira Kuanyshbekova	Critical	2027-03-01
SCI-29	Requirement checks (rules + intelligent) and topic similarity; E5	Saltanat Sergeyeva	Critical	2027-03-20
SCI-30	Integration, E2E tests, performance (E6), usability (E7)	Meruert Khairgeldinova	Critical	2027-04-15

11. Supervisor Feedback

Supervisor: Almas Myrzakhmetov. Every comment gets a decision and a trace (YouTrack task / PR) so the committee can see that review led to changes.

#	Date	Deliverable	Comment (supervisor)	Decision (accepted / partly / rejected + reason)	Task / PR	Status
1	13.10.2026	Justification letter (source: supervisor's «Обоснование»)	Responsibilities split by system layers; topic archive to be requested from the School at the requirements stage; bi-weekly control meetings	Accepted — reflected in Sections 1 and 10 (D1, R1)	SCI-1	Done

Further comments on the proposal, requirements, architecture, prototype and verification plan (SCI-2…SCI-13) are logged here after each bi-weekly meeting; key comments from pull requests are copied with the PR link.

Appendix A. Architecture Decision Records

ADR-001: Server on Node.js + Express

- **Status:** Accepted · **Date:** 22.10.2026 · **Owner:** M. Khairgeldinova · reviewed by supervisor
- **Context:** The server handles upload, storage, link/DOI checking (I/O-bound, many concurrent requests) and orchestration of checks. ML code lives in a separate Python service (ADR-003). Alternatives: FastAPI, NestJS (Section 5, D1).
- **Decision:** Node.js (LTS) + Express; multer (upload), joi/zod (validation), Mongoose, swagger-jsdoc + swagger-ui (OpenAPI), undici/axios with concurrency limit (p-limit) for link checks; Jest + supertest for tests.
- **Consequences:** + matches team expertise, efficient async I/O. − two languages in the system -> contracts must be explicit (OpenAPI, shared JSON schemas in docs/api/).

ADR-002: MongoDB as main storage

- **Status:** Accepted · **Date:** 22.10.2026 · **Owner:** M. Khairgeldinova
- **Context:** Check reports are nested and evolve as checks are added; requirement templates are user-defined. Alternatives: PostgreSQL, PostgreSQL+JSONB (Section 5, D2).
- **Decision:** MongoDB with collections users, documents, checkJobs, reports, requirementTemplates, topics; files in GridFS; Mongoose schemas enforce structure. Topic embeddings are kept in the analysis service index, rebuilt when topics changes.
- **Consequences:** + flexible reports, team familiarity. − referential integrity in application code; indexes on documents.userId, checkJobs.status required.

ADR-003: REST between components, asynchronous check jobs

- **Status:** Accepted · **Date:** 22.10.2026 · **Owners:** whole team
- **Context:** Three components are developed in parallel by different people in different languages; some checks take minutes. Alternatives: synchronous REST, message broker (Section 5, D3).
- **Decision:** Client <-> Server: REST/JSON. Server <-> Analysis service: REST/JSON; server creates a checkJob, calls POST /analyze/{check} on the analysis service, which returns 202 and later posts results to POST /internal/jobs/{id}/result (shared secret). Client polls GET /api/checks/{id}. Contracts in docs/api/openapi-*.yaml, agreed before coding (task SCI-9).
- **Consequences:** + independent development and testing with mocks; no extra infrastructure. − must handle lost callbacks (timeout -> retry/failed status).

ADR-004: Hybrid document structure extraction

- **Status:** Proposed (final after experiment E1) · **Owners:** M. Khairgeldinova (upload, text extraction), S. Sergeyeva (reference/citation parsing)
- **Context:** Reference lists in AITU theses use GOST, APA and IEEE styles; documents are DOCX or PDF.
- **Decision:** DOCX -> python-docx/mammoth; PDF -> GROBID (TEI) with PyMuPDF fallback; reference strings -> GROBID parser + rules for GOST; in-text citations -> regex for [n], (Author, Year), [n, p. x].
- **Consequences:** + best available parser for PDFs (Tkaczyk et al. 2018). − Java service in Docker Compose; GOST quality must be measured (E1, target F1 >= 0.85).

ADR-005: NLP approach for citation, abstract and topic checks

- **Status:** Proposed (model chosen after E3–E5) · **Owner:** S. Sergeyeva
- **Context:** Cross-lingual (RU/KZ/EN), paraphrase-aware comparison is required; documents are confidential.
- **Decision:**
  - Citation check: split source text into passages -> retrieve top-k by multilingual embeddings (candidates: multilingual-e5-base, LaBSE, paraphrase-multilingual-mpnet) -> NLI cross-encoder label (supported / partial / not supported) -> show fragment and score. Baseline: TF-IDF cosine.
  - Abstract check: summary (LLM with consent, else extractive) + coverage (BERTScore-style) + NLI consistency of each abstract sentence against the body.
  - Topic check: embedding of title + abstract vs. archive; top-5 with cosine score; threshold calibrated on labelled pairs.
  - LLM: provider-agnostic interface; disabled unless the user consents per document.
- **Consequences:** + local, explainable, multilingual. − Kazakh quality uncertain -> reported separately in evaluation.

Appendix B. Survey Questionnaire and Interview Guide

B.1 Survey S-1 (Google Forms, 23.10.2026, N = 26)

1. What is your current role? (undergraduate student; master's student; PhD student; university lecturer or professor; other)
2. How often do you check references and citations in academic documents? (always; often; sometimes; rarely; never)
3. How do you usually check references and citations in scientific documents? (manually; open links and verify sources individually; reference management software; plagiarism detection software; other digital tools; do not usually check; other)
4. How much time do you usually spend checking references and citations in one scientific document? (less than 15 min; 15–30 min; 31–60 min; more than 1 hour; depends on the document; do not perform these checks)
5. What is the most difficult part of checking an academic document? (broken links or incorrect DOIs; whether sources actually support citations; abstract accuracy; compliance with academic requirements; similar research topics; other)
6. Have you ever encountered broken links or references that could not be verified? (yes; no; not sure)
7. What tools do you currently use to check academic documents? (Turnitin or similar; DOI or reference lookup services; reference management tools; manual checking; other)
8. Which features would be most useful to you in such a system? (link check; DOI and bibliographic metadata; citations support statements; abstract vs. content; academic requirements; similar research topics; report with detected issues; other)
9. What additional features or improvements would you suggest? (open question)

B.2 Interview guide (I-1…I-3, before pre-defense 2)

Explain purpose; ask consent for notes; agree on anonymity.

Context

10. Your role? How many theses / articles do you check or write per year?
11. What do you check first in a thesis? What takes the most time?

References

3. How do you check the reference list? Do you open links? How often are they broken or wrong?
4. Have you seen references that do not exist (e.g. generated by ChatGPT)? How did you notice?

Citations

5. Do you check whether a quotation or paraphrase matches the source? How often do you find mismatches?

Abstract

6. How often does the abstract not reflect the content? What is typically wrong?

Requirements

7. Which formal requirements do you check (structure, volume, number/age of references, formatting)? Are they the same for all programmes?

Topics

8. How is a repeated topic detected today? Has it happened?

Tools

9. Which tools do you use (Turnitin, StrikePlagiarism, others)? What do they not cover?
10. What would make you trust an automatic report? What must it show?
11. If the system could do only three things — which?

Protocol template (docs/evidence/I-n.md)

```
ID | Date | Role | Duration | Interviewer
Key quotes:
Pain points:
Measured values (time, counts):
Claims confirmed / contradicted (C#):
New requirements:
```